

Input and output

A hands-on course in building agents by building their harness

Lesson 2

Where we left off: both ends were stubs

- Going in: a hardcoded question no one typed
- Coming out: a raw dump of the response
- The model said to run `ls` , and nothing could do it
- A model only takes tokens in and gives tokens out: it can't read what you type or run a command

Two pathways around the call

person or world → input → request → model interface → response → output → person or world

- **Input** gathers inputs: from a person (a message, a question, a correction) or from the world (what happened when a command ran)
- **Output** handles the model's outputs: shown to a person, or run as a tool
- Input is afferent, carrying signals in; output is efferent, carrying actions out

The model asks; the harness acts

- The exchange runs through **tools**
- The model can't act, but its response can include a **request** to use a tool: a name and arguments, in a shape the harness described
- Output runs the request; what happened comes back as input
- Without tools, the only thing a model can change is what a person reads

Input: input.py adds read() (abridged)

```
# — input —  
def read(prompt):                                # input: from a person  
    while True:  
        print(prompt, end="", flush=True)  
        line = sys.stdin.readline()  
        if not line: return "/q"                # ...  
        if line.strip(): return line.rstrip("\n") # ...  
        prompt = "> "  
input = " ".join(sys.argv[1:]) or read("> ")  
if input == "/q": sys.exit()
```

Comments cut. The output is still Lesson 1's stub: the whole response, dumped.

Input is whatever comes in

- `input` isn't assumed to be a task or a question; it goes in as the `user` message
- `read()` prints a prompt and reads a line: the afferent pathway from a person
- Enter on an empty line gives a fresh `>`, as a terminal does; nothing is sent
- End of input (Ctrl-D) returns `/q`, so the harness can stop
- The new `# — output: the one tool —` section describes `bash` in the request; nothing can run it yet

Run it: the model asks, nothing acts

```
uv run lessons/02-input-and-output/input.py "how many lines are in README.md?"
```

```
    "input": {  
      "cmd": "wc -l README.md"  
    },  
    "name": "bash",  
    "type": "tool_use",  
    "stop_reason": "tool_use",
```

One real run (shortened)

Output: output.py shows and runs (abridged)

```
def show(event):                                # output: text, shown as it's written
    if event.type == "text": print(event.text, end="", flush=True)
    if event.type == "content_block_stop" and event.content_block.type == "text": print()

output = call(show, max_tokens=16384, tools=tools, messages=[...]).content

for block in output:                            # output: run tool requests
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, ...)
        print(done.stdout)
```


Each block goes where it belongs

- `tools` is what output can do: a name, a description the model reads, a schema for arguments
- `show()` : `call(show, ...)` hands it each streamed piece; text is printed the moment it arrives
- `output` is now `call(...).content` , the whole response once it's done
- Its text has already been shown, so the loop only runs `tool_use` blocks; thinking goes nowhere
- `max_tokens` is 16384 from now on, so a response that thinks first has room to finish a tool request

Run it: output runs the command

```
uv run lessons/02-input-and-output/output.py
```

```
$ ls -la
total 192
-rw-r--r--  1 root root 28138 Oct  6 20:31 README.md
drwxr-xr-x  6 root root  4096 Oct  5 19:49 lessons
-rw-r--r--  1 root root   246 Oct  6 03:33 pyproject.toml
```

One real run (shortened). The listing went to the screen, and nowhere else.

Together: quark.py (abridged)

```
output = call(show, max_tokens=16384, tools=tools, messages=[...]).content

input = []
for block in output:                                     # output: run tool requests
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, ...)
        print(done.stdout)
        input.append({"type": "tool_result", ..., "content": ...}) # input: from the world
```

Before these lines, quark.py is input.py plus output.py's `show()`. Lesson 1's dump is gone.

input = [] is input from the world

- What a tool printed is something the model should know about
- Each `tool_result` names the `tool_use_id` it answers
- A command that prints nothing still reports its exit code
- Same name as input from a person, on purpose: input is whatever comes in

Run it: the exchange, one way

```
uv run lessons/02-input-and-output/quark.py "how many lines are in README.md?"
```

```
$ wc -l README.md  
301 README.md
```

- Same input as input.py, same `wc -l`. This time output ran it
- The model never sees the 301. It's sitting in `input`, and nothing sends it

What else input and output can be

- **Input from a person:** command line, a prompt, a pipe, a chat app, a webhook, a schedule; text, images, files
- **Input from the world:** what a tool printed, its exit code, whether it failed
- **Output:** a terminal, a file, a chat message, a pull request; all at once or as produced
- **Tools:** one general tool or many routed by name; run here, in a container, remotely; one by one or all at once
- **A product:** MCP servers package tools behind one protocol, output you plug in

input_output.py: Telegram, tools at once (abridged)

```
async def execute(block, cut_off):
    if cut_off:
        out, failed = "cut off before it was finished, so it was not run", True
    elif block.name not in executors:
        out, failed = f"no tool named {block.name}", True
    else:
        out, failed = await executors[block.name](block.input)
    return {"type": "tool_result", "tool_use_id": block.id, ...}
```

- `receive()` takes a Telegram message; `send()` posts back to the chat
- `executors` routes `bash` and `read_file` by name
- `asyncio.gather` runs every tool request at once

One message, three tools, run together

Sent: "check the python3 version, the git version and the first line of pyproject.toml, all three at once"

```
[bot → telegram] → bash {"cmd": "python3 --version"}  
Python 3.13.14  
[bot → telegram] → bash {"cmd": "git --version"}  
git version 2.43.0  
[bot → telegram] → bash {"cmd": "head -n 1 pyproject.toml"}  
[project]
```

One real run (shortened), Telegram swapped for a stand-in that prints each message; the model and tools were real

The rule

- **Input** gathers what goes to the model, from a person or from the world
- **Output** handles what comes back, shown to a person or run as a tool
- A tool is the model asking and the harness acting

What they never do

- Getting the request there and the response back is the **model interface**
- When to call, and whether a result goes back around, is **control flow**
- How what input gathers is presented in the request is **context**
- Input only gathers; output only handles the response

The path ends at output

- The model asked for `wc -l`, it ran, and the model never saw what it found
- Something has to send the result back to the start and decide to go again
- That's **control flow**

Next: Control flow

A hands-on course in building agents by building their harness

Lesson 3